

Supplementary Material

Towards Distinguishing False Positives from Genuine Regressions in Optimisation-Aware GCC Plugin Testing

Anonymous

Anonymised for double-blind review

This document provides extended tables and figures supporting the main paper. It is provided as supplementary material and is not required to follow the main argument. Section and table numbering here are independent of the main paper.

1 Pipeline and Report Generation

The data studied in the main paper is produced by a white-box differential testing framework for GCC plugins. This section describes how that framework generates the per-variant outcomes and how the six-stage investigator automatically resolves most of them, so that the counts reported in the main paper can be interpreted in context.

1.1 Data Generation

The framework produces outcomes in three phases. First, it clones each plugin’s source tree and injects structured logging using a libclang-based AST pass. For every function that passes an 11-check safety gate, it inserts lightweight `LOG_ENTER` and `LOG_EXIT` calls that emit schema-versioned JSON events carrying function identity, source location, event type, and parameter and return summaries. Instrumentation operates only on cloned sources, so production plugin code is unchanged, and the injected calls carry observable file-I/O side effects, so the compiler cannot eliminate them even when the enclosing function is inlined.

Second, it compiles each instrumented plugin under 15 optimisation variants (spanning optimisation levels, inlining budgets, LTO, vectorisation, and alias analysis) plus a `-g` BASE build, and executes each variant against the same high-coverage test corpus. Each (plugin, test run, variant) triple yields one outcome; across 17 plugins and 69 test runs this gives the 1,163 outcomes analysed in the main paper.

Third, for every variant it performs two complementary comparisons against BASE: a *binary diff* (using `nm` for symbol changes and `objdump` for disassembly) and a *behavioural diff* that reconstructs call frames from the JSON traces and compares call-frame counts, call sequences, and parameter and return summaries. Each variant receives an initial verdict: `OK`, `NOISY_ONLY`, `SUSPICIOUS` (binary changes but no trace divergence), or `CANDIDATE_DEVIATION` (trace divergence not

explained by benign optimisation). Only SUSPICIOUS and CANDIDATE__DEVIATION variants proceed to the investigator.

1.2 Six-Stage Investigator

The investigator refines each escalated variant using progressively richer evidence, resolving benign cases early so that only genuinely ambiguous outcomes reach the expensive runtime stage.

1. **Symbol disappearance analysis.** Missing call-frame entries are correlated with `nm` symbol-table changes. A disappearance explained by inlining or dead-code elimination is tentatively benign; one without a structural explanation is escalated.
2. **CFG comparison.** Control-flow graphs of affected functions are compared via `objdump`. Differences consistent with loop unrolling, vectorisation, or block reordering are tagged as benign restructuring.
3. **Statistical profiling.** Event counts, call frequencies, and return-value distributions are compared. Benign optimisation typically preserves these statistics; divergences indicate a likely behavioural change.
4. **Cross-variant correlation.** A deviation appearing under a single variant is more likely noise; one recurring across independent optimisation axes is more likely meaningful and is prioritised.
5. **Semantic tagging.** Accumulated evidence is summarised as safe or bug-indicating tags (Table ?? of the main paper), which determine whether a variant is discharged or escalated.
6. **Runtime confirmation.** The most suspicious variants are re-executed and their instrumented-function sets compared against BASE. A strict subset indicates the plugin failed to instrument functions it should have (confirmed deviation); an identical set overrides the static verdict as confirmed benign. This stage depends on being able to load the instrumented plugin into a matching GCC instance, which fails under a compiler ABI mismatch (Lesson 5 of the main paper).

1.3 Report Contents

For each non-trivial outcome the framework emits a compact report used for developer triage. Each report contains the binary-diff summary (symbols added or removed, size deltas), representative BASE-versus-variant trace excerpts, the semantic tags assigned, the cross-variant recurrence pattern, any runtime-confirmation result, and a root-cause hypothesis. These reports are the artefacts aggregated into the tables and figures throughout this document, and are released in full in the repositories listed under Data Availability.

2 Full Corpus

Table 1 lists all 17 plugins in the corpus, their category, source, and the number of test programs used. Test programs were generated using a search-based technique

that iteratively mutates candidate programs and retains those that improve branch coverage of the plugin’s instrumentation logic.

Table 1. Plugin corpus: 17 plugins across five categories.

Plugin	Category	Source	Test programs
stackleak	Security	Linux kernel	4
latent_entropy	Security	Linux kernel	4
randomize_layout	Security	Linux kernel	4
arm_ssp_per_task	Security	Linux kernel	4
arm_ssp	Security	Linux kernel	4
SecRetAddress	Security	Standalone	4
structleak	Security	Linux kernel	4
sancov	Analysis	Standalone	4
cfed	Analysis	Standalone	5
oi	Analysis	Standalone	4
cprintf	Utility	Standalone	4
gcc_assert_intr	Utility	Standalone	4
funcp-encrypt	Obfuscation	Standalone	4
rofirrim_01	Experimental	Standalone	4
rofirrim_02	Experimental	Standalone	4
rofirrim_03	Experimental	Standalone	4
rofirrim_04	Experimental	Standalone	4

3 Per-Plugin Escalation Breakdown

Table 2 shows the per-plugin candidate-deviation breakdown across all 12 affected plugins, with the maximum number of co-occurring bug-indicating tags and the dominant risk. Five plugins (SecRetAddress and the four rofirrim experimental plugins) produced no escalations and are omitted.

4 False-Positive Reduction Funnel

Table 3 shows how the 1,163 optimisation-variant outcomes are progressively resolved across the pipeline stages, and Figure 1 visualises the same reduction. The binary filter closes non-structural changes, static tagging discharges a predictable class, and runtime confirmation resolves the largest single block of benign cases. The residual 630 outcomes form the escalation class.

5 Fragility Index

To compare plugins independently of variant count, we define a Fragility Index as the number of candidate-deviation findings divided by the total number of

Table 2. Candidate-deviation findings per plugin (12 of 17 plugins affected). CD/total is the number of escalated variants over total variants. Max tags is the maximum number of co-occurring bug-indicating tags across all 15 variants.

Plugin	Category	CD/total	Max tags	Dominant risk
stackleak	Security	15/15	10	Gate and tracking functions restructured
latent_entropy	Security	15/15	10	22 core functions inlined away
randomize_layout	Security	13/15	8	Struct randomisation pass altered
funcp-encrypt	Obfuscation	15/15	10	Encryption logic fully inlined
gcc_assert_intr	Utility	14/15	7	Assert inlining with complex logic
arm_ssp	Security	8/15	5	Gate and execute restructured
sancov	Analysis	9/15	6	Coverage hook insertion logic
arm_ssp_per_task	Security	7/15	1	plugin_init complexity growth only
cprintf	Utility	7/15	4	Complexity and inlined logic in print pass
oi	Analysis	13/15	3	Inlined complex logic only
cfed	Analysis	6/15	2	Complexity change only
structleak	Security	5/15	3	Moderate complexity across passes

Table 3. False-positive reduction through the triage pipeline. Each row reports the outcomes resolved at that stage, the cumulative resolved count, and the number still open.

Stage	Resolved at stage	Cumulative resolved	Still open
Binary filter (noisy/ok)	143	143 (12.3%)	1,020
Static tagging (discharged)	90	233 (20.0%)	930
Runtime confirmation (benign)	300	533 (45.8%)	630
Residual escalation class	—	—	630 (54.2%)

optimisation-variant outcomes tested for that plugin. An index near 1 indicates that unexplained behavioural differences were observed under almost all tested variants. Figure 2 plots the index for all 17 plugins.

6 stackleak Gate Growth

Table 4 shows the instruction growth of the `stackleak_gate` function across variants. The O2-to-O3 jump is non-linear, indicating qualitative restructuring rather than proportional growth. The gate is not flagged at `O0` or `NO_INLINE`, where aggressive inlining does not occur.

7 Coverage Versus Finding Severity

Table 5 illustrates, for one plugin with four test programs, how coverage relates to finding severity. Lower coverage produces escalations with fewer co-occurring bug-indicating tags because fewer plugin functions are exercised. This is a single-plugin observation and is not claimed to generalise without further study.

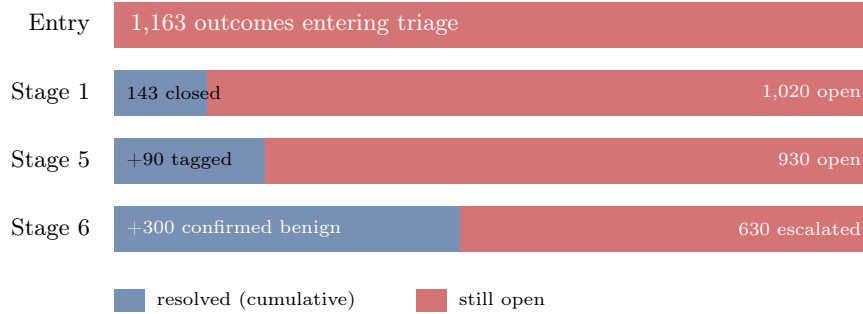


Fig. 1. False-positive reduction funnel. Blue segments show the cumulative outcomes resolved without developer intervention (45.8% by Stage 6); red segments show those still open. The final 630 outcomes (54.2%) form the escalation class.

Table 4. `stackleak_gate` instruction growth by variant. Added/removed counts are relative to the debug baseline.

Variant	Added/removed	Bug tags	Notes
O1	+249/−130	6	Complexity-change onset
O2	+267/−130	8	Modest increase
O3	+978/−130	10	Non-linear jump
O3_LOOPS	+979/−130	10	Same as O3
OS	—	5	Gate not flagged at Os
NO_INLINE	—	5	Gate not flagged without inlining
LTO	+978/−130	10	Same as O3
INLINE_ALL	+978/−130	10	Same as O3

8 Triage Funnel by Variant Type

Table 6 breaks the reduction funnel down by selected variant type. The most aggressive strategies (O3, LTO, `INLINE_ALL`) yield the fewest static resolutions because they absorb logic-bearing functions. `NO_INLINE` is the instructive counterpoint: disabling inlining leaves only always-inline header functions, all resolved statically.

9 Suspiciousness Ranking Illustration

The triage workflow ranks escalated variants by a weighted combination of (i) the number of bug-indicating tags, (ii) net instruction growth in gate and execute callbacks (weighted more heavily than auxiliary functions), (iii) the presence of `VOLATILE_RETURN`, and (iv) recurrence of similar risk signals across optimisation variants. Table 7 illustrates how these signals separate variant types using the aggregate data of Table 6: the most aggressive strategies concentrate the highest residual escalation counts and the fewest static resolutions, and are therefore

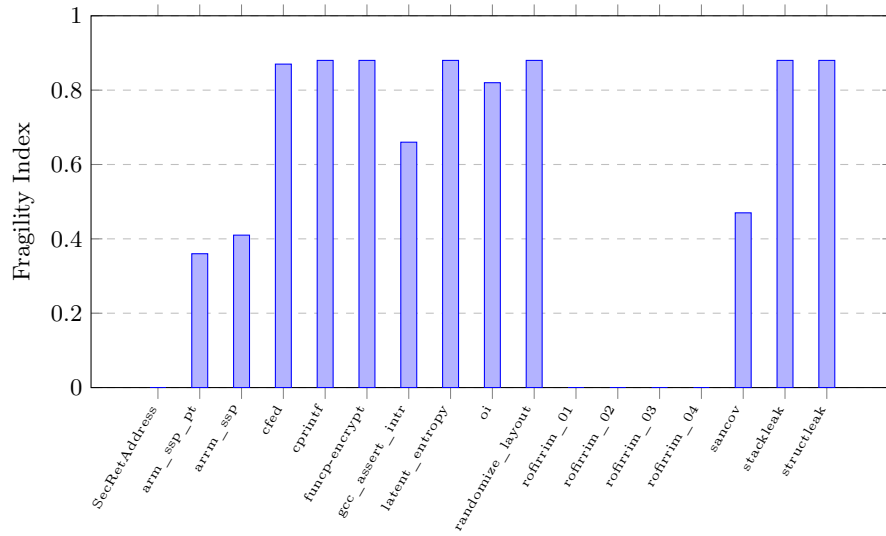


Fig. 2. Fragility Index per plugin: candidate deviations over total optimisation-variant outcomes. Experimental plugins and SecRetAddress show an index of 0; analysis, obfuscation, and several security plugins approach 0.9.

Table 5. Coverage versus finding severity (one plugin, four test programs). Max tags is the maximum across all 15 variants.

Test program	Coverage	Max tags	Notes
test_1	72.2%	10	All three tags, all 15 variants
test_2	70.1%	10	All three tags, all 15 variants
test_3	18.2%	6	Complexity-change only, fewer variants
test_4	62.4%	8	All three tags in most variants

ranked highest for selective runtime confirmation. Selecting the top three variant types by residual count (LTO, INLINE_ALL, O3) captures the cross-translation-unit and aggressive-inlining classes that are hardest to resolve statically. We treat this ranking as an operational heuristic; a formal evaluation against simpler baselines (e.g., instruction-growth-only ranking) is left to future work.

10 Log-Binary Divergence Example

Table 8 shows one security plugin whose execution profile is identical across all 15 optimised variants despite 20–22 symbols being removed by inlining. This motivates combining binary and log evidence: aggressive inlining can relocate injected log events without altering their execution, so the two oracles disagree and neither alone is sufficient.

Table 6. Triage funnel by selected variant type. Binary flags is the number of plugins flagged under that variant; static resolved is the number closed by static tagging; residual is the number escalated.

Variant	Binary flags	Static resolved	Residual	Primary cause of residual
O1	17	5	12	Helper inlining onset at O1
O2	17	8	9	Branch elimination and conservative inlining
O3	17	2	15	Aggressive inlining crosses threshold
LTO	17	1	16	Cross-TU inlining beyond static resolution
INLINE_ALL	17	1	16	Maximum inlining budget
NO_INLINE	15	9	6	Header-defined always-inline only

Table 7. Illustrative ranking of variant types by residual escalation load, derived from Table 6. Higher residual count and lower static resolution indicate higher priority for runtime confirmation.

Rank	Variant	Residual escalations	Static resolution rate
1	LTO	16	Low (1/17)
2	INLINE_ALL	16	Low (1/17)
3	O3	15	Low (2/17)
4	O1	12	Moderate (5/17)
5	O2	9	Moderate (8/17)
6	NO_INLINE	6	High (9/15)

11 Data Availability

All artefacts used in this study are available at <https://anonymous.4open.science/r/Logger-106B> and https://anonymous.4open.science/r/diff_testing_gcc_plugin_reports-D257.

Table 8. Log–binary divergence: identical execution profiles despite 20–22 symbols removed (one security plugin, all 15 optimised variants).

Variant group	Symbols removed	Log frames	Log events	Issues
DEBUG / O0 (baseline)	0	53	231	0
O1	20	53	231	0
O2, OS, NO_INLINE	21	53	231	0
O3, LTO, INLINE_ALL	22	53	231	0